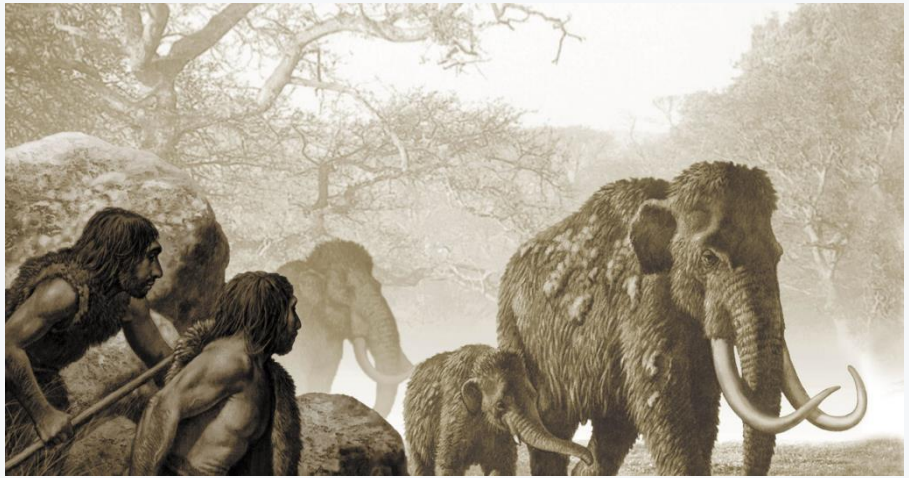
saby

Валим «слона»

Highload «из ниоткуда»: проблемы архитектуры клиентского приложения, которые «убивают» PG



спикер

Кирилл Боровиков

Компания «Тензор», технический директор

explain.tensor.ru, saby.ru, sbis.ru

https://github.com/Kilor/Presentations

saby

Кирилл Боровиков

- 25+ лет в разработке, почти 20 лет с PostgreSQL
- «Тензор» - сотни проектов, тысячи разработчиков
- технический директор, ведущий архитектор БД
- 150+ хабрапубликаций (@Kilor)
- explain.tensor.ru – анализируем ваши планы

saby

О чем поговорим?

- «нет!» «черным ящикам»

  зачем знать, как что устроено «внутри»

- ресурсы не бесконечны

  особенно, когда их все хочет ИИ

- без[д]умный SQL

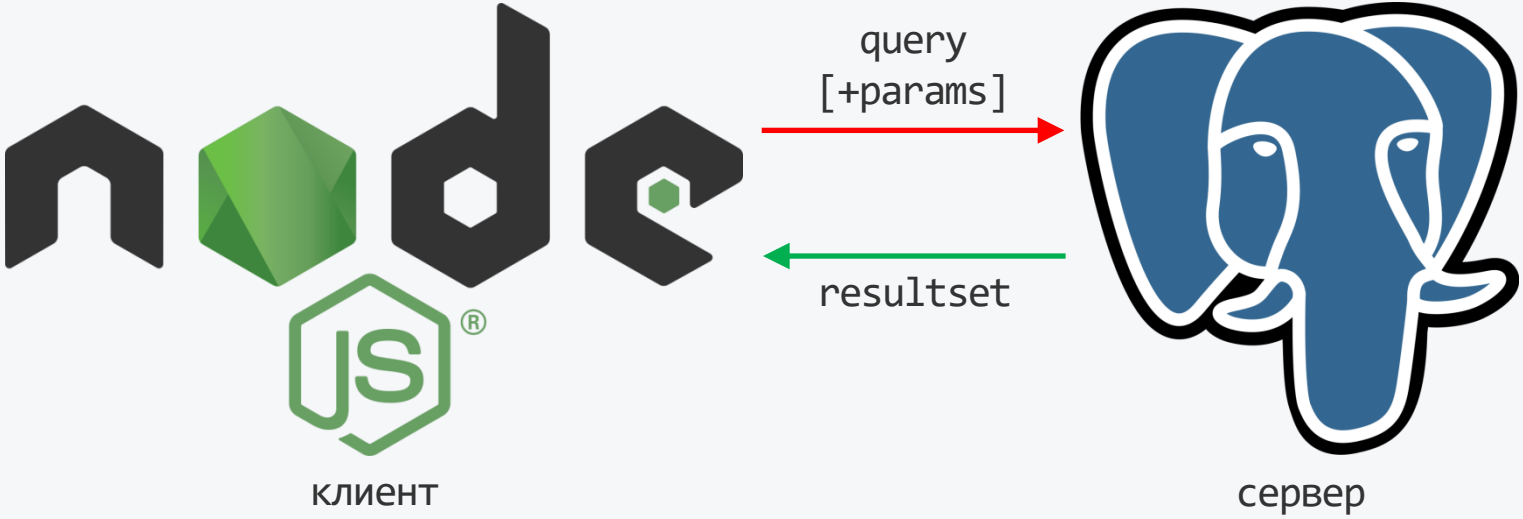
  не стоит слепо доверять ORM и «джунам»

- размер имеет значение

  знай, что и зачем передаешь

saby

Тестовый полигон



saby

Тестовый полигон (Node.js)

```
const pg = require('pg');
const {Client} = pg;
const connInfo = {
  user     : 'postgres'
, password : 'postgres'
, host     : 'localhost'
, port     : 5432
, database : 'postgres'
};

const jobs = 1; // количество одновременных соединений
```

saby

Тестовый полигон (Node.js)

```
(async () => {
  // подключаем к базе всех клиентов
  const clients = await Promise.all(
    Array(jobs).fill().map(async () => {
      const client = new Client(connInfo);
      await client.connect();
      // активируем auto_explain в «параноидальном» режиме
      await client.query(`
LOAD 'auto_explain';
SET auto_explain.log_analyze = 'on';
SET auto_explain.log_buffers = 'on';
SET auto_explain.log_min_duration = 0;
SET auto_explain.log_nested_statements = 'on';
SET auto_explain.log_timing = 'on';
SET auto_explain.log_triggers = 'on';
      `);
      return Promise.resolve(client);
    })
  );
```

saby

Тестовый полигон (Node.js)

```
  // одновременно выполняем на каждом клиенте необходимые запросы
  const rows = await Promise.all(
    clients.map(async client => {
      const res = await client.query('SELECT 1'); // тестируемый запрос
      return Promise.resolve(res.rows);
    })
  );

  // закрываем все соединения, когда все выполнили
  await clients.forEach(client => client.end());
})();
```

saby

Тестовый полигон (PostgreSQL)

```
2026-03-01 13:37:13.173 MSK [29048] LOG:  duration: 0.015 ms  plan:
  Query Text: SELECT 1
  Result  (cost=0.00..0.01 rows=1 width=4) (actual time=0.004..0.007 rows=1 loops=1)
```

Explain / Explain Analyze | имя плана в архиве... | Copy&Paste or Drag'n'Drop ↓

для получения информации о распределении данных в памяти и времени работы с диском используйте

```
SET track_io_timing = TRUE; EXPLAIN (ANALYZE, BUFFERS) { SELECT | INSERT | UPDATE | DELETE } …
```


```
2026-03-01 13:37:13.173 MSK [29048] LOG:  duration: 0.015 ms  plan:
  Query Text: SELECT 1
  Result  (cost=0.00..0.01 rows=1 width=4) (actual time=0.004..0.007 rows=1 loops=1)
```

saby

Тестовый полигон (PostgreSQL)

```
2026-03-01 13:37:13.173 MSK [29048] LOG:  duration: 0.015 ms  plan:
  Query Text: SELECT 1
  Result  (cost=0.00..0.01 rows=1 width=4) (actual time=0.004..0.007 rows=1 loops=1)
```

#	node, ms	tree, ms	rows	Result
		.007	1	итоговые результаты (4B = rows=1 x width=4)
0	.007		1	Result (cost=0.00..0.01 rows=1 width=4) (actual time=0.004..0.007 rows=1 loops=1)

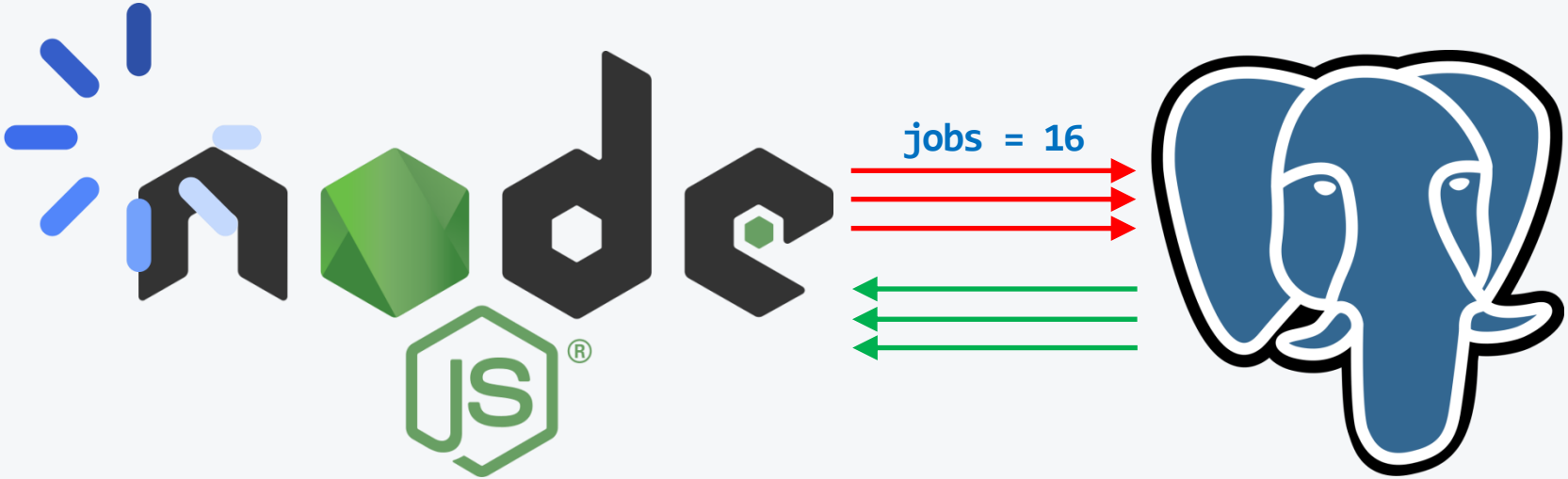
время получения ответа **на клиенте**

объем переданного **трафика**

время выполнения запроса **на сервере**

saby

Проблема на клиенте



saby

Проблема на клиенте (обработка ответа)

```
SELECT i FROM generate_series(1, 1e6) i; -- миллион чисел
```

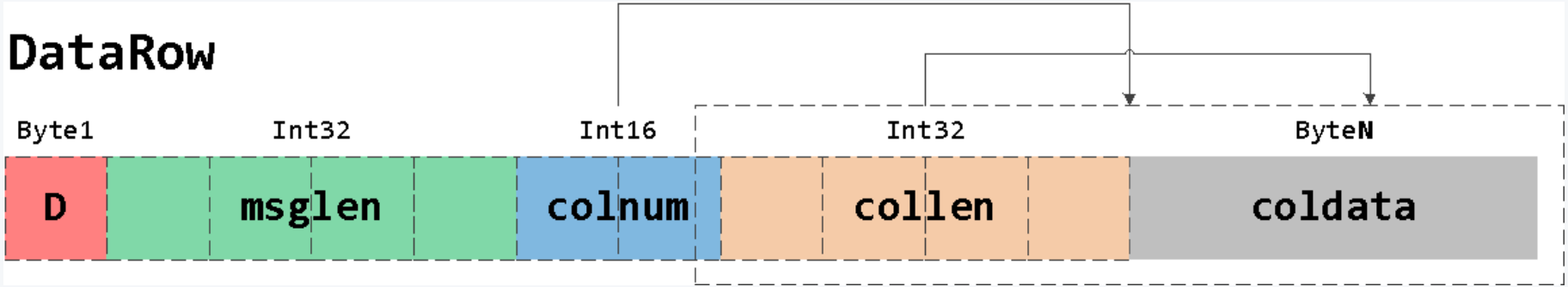
```
... 16:31:38.570 MSK [13836] LOG:  duration: 855.744 ms  plan:
  actual time=..409.293 ...
... 16:31:39.201 MSK [19448] LOG:  duration: 1383.853 ms  plan:
  actual time=..413.909 ...
... 16:31:40.321 MSK [38264] LOG:  duration: 1515.595 ms  plan:
  actual time=..435.973 ...
... 16:31:40.366 MSK [41512] LOG:  duration: 1560.709 ms  plan:
  actual time=..447.622 ...
... 16:31:42.032 MSK [24476] LOG:  duration: 2600.114 ms  plan:
  actual time=..579.276 ...
... 16:31:42.124 MSK [41900] LOG:  duration: 2692.630 ms  plan:
  actual time=..549.336 ...
... 16:31:44.588 MSK [39248] LOG:  duration: 3350.654 ms  plan:
  actual time=..773.811 ...
... 16:31:44.878 MSK [40748] LOG:  duration: 3617.941 ms  plan:
  actual time=..893.361 ...
```

```
... 16:31:45.411 MSK [35176] LOG:  duration: 4359.566 ms  plan:
  actual time=..801.552 ...
... 16:31:45.505 MSK [12416] LOG:  duration: 4453.663 ms  plan:
  actual time=..863.834 ...
... 16:31:45.693 MSK [23112] LOG:  duration: 4455.087 ms  plan:
  actual time=..863.590 ...
... 16:31:46.128 MSK [36044] LOG:  duration: 4890.108 ms  plan:
  actual time=..947.209 ...
... 16:31:49.449 MSK [38768] LOG:  duration: 6888.474 ms  plan:
  actual time=..696.957 ...
... 16:31:49.541 MSK [35268] LOG:  duration: 6980.557 ms  plan:
  actual time=..667.114 ...
... 16:31:49.632 MSK [18764] LOG:  duration: 7071.027 ms  plan:
  actual time=..698.535 ...
... 16:31:49.720 MSK [28356] LOG:  duration: 7135.465 ms  plan:
  actual time=..690.181 ...
```

10x

saby

Проблема на клиенте (обработка ответа)

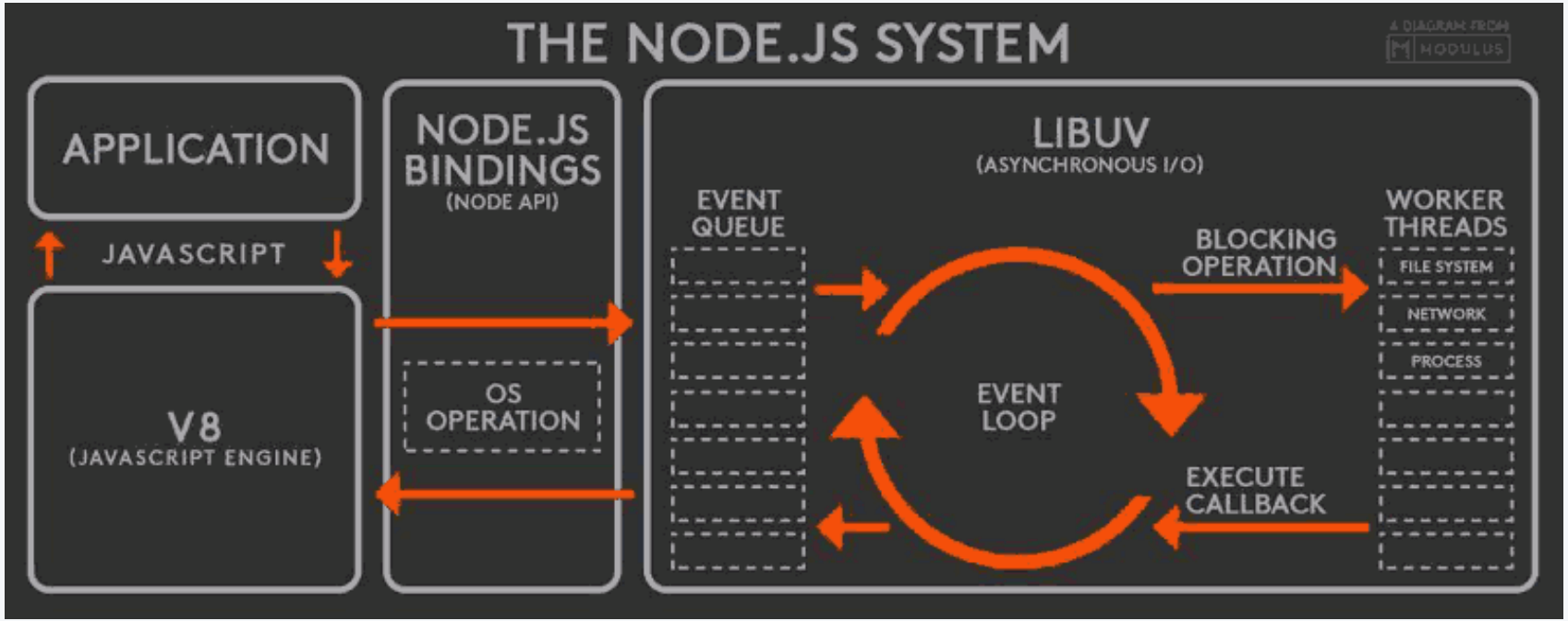


и так – каждая запись серверного ответа

https://www.postgresql.org/docs/current/protocol-message-formats.html#PROTOCOL-MESSAGE-FORMATS-DATAROW

saby

Проблема на клиенте (обработка ответа)

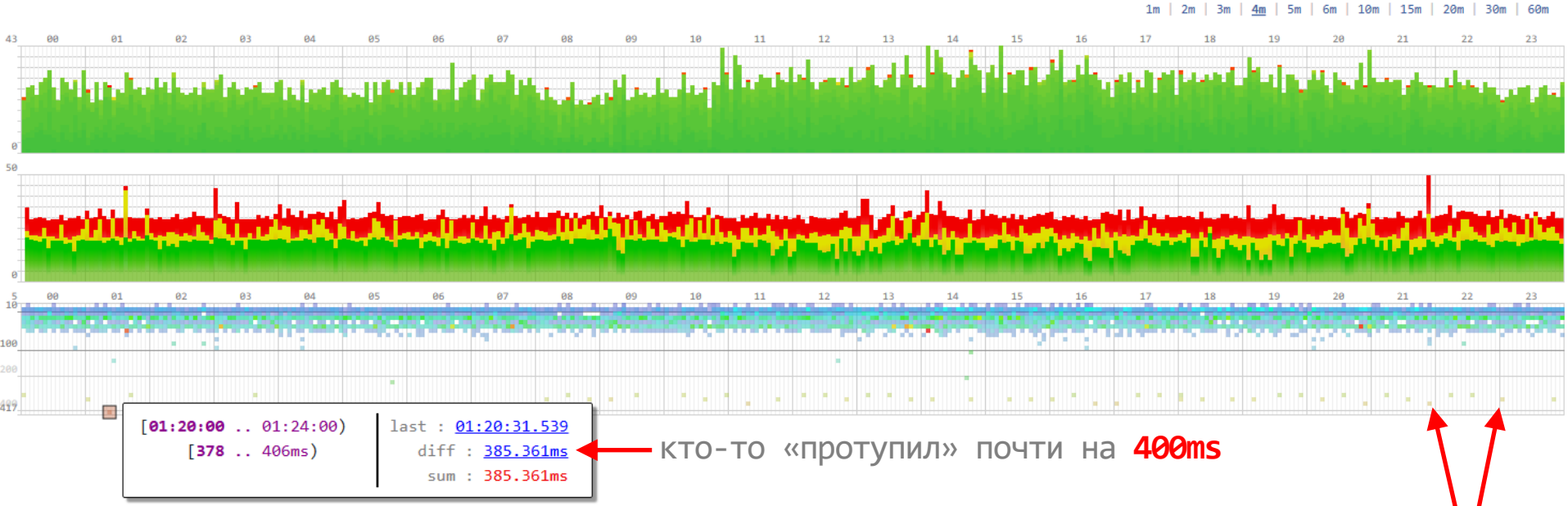


Event Loop однопоточен и «един на всех»

The Node.js Event Loop

saby

Проблема на клиенте (обработка ответа)



кто-то «протупил» почти на **400ms**

… и поступает так **регулярно**

saby

Проблема на клиенте (обработка ответа)

```
2026-03-09  01:20:31.539  MSK  [2324966:34/303350586]  [contract-service]  6903253552625005045  ()
<application instance> : <method name> LOG:  duration 597.423 ms  plan:
        Query Text: <query text>
        HashAggregate  (cost=2813.42..2813.55 rows=9 width=48) (actual time=204.370..212.062 rows=9
loops=1)
          ...
```

пострадавший

проблема!

записей совсем немного

saby

Проблема на клиенте

- блокировка Event Loop / GIL / …

  много [непрерывной] вычислительной нагрузки

- исчерпание ресурсов процесса

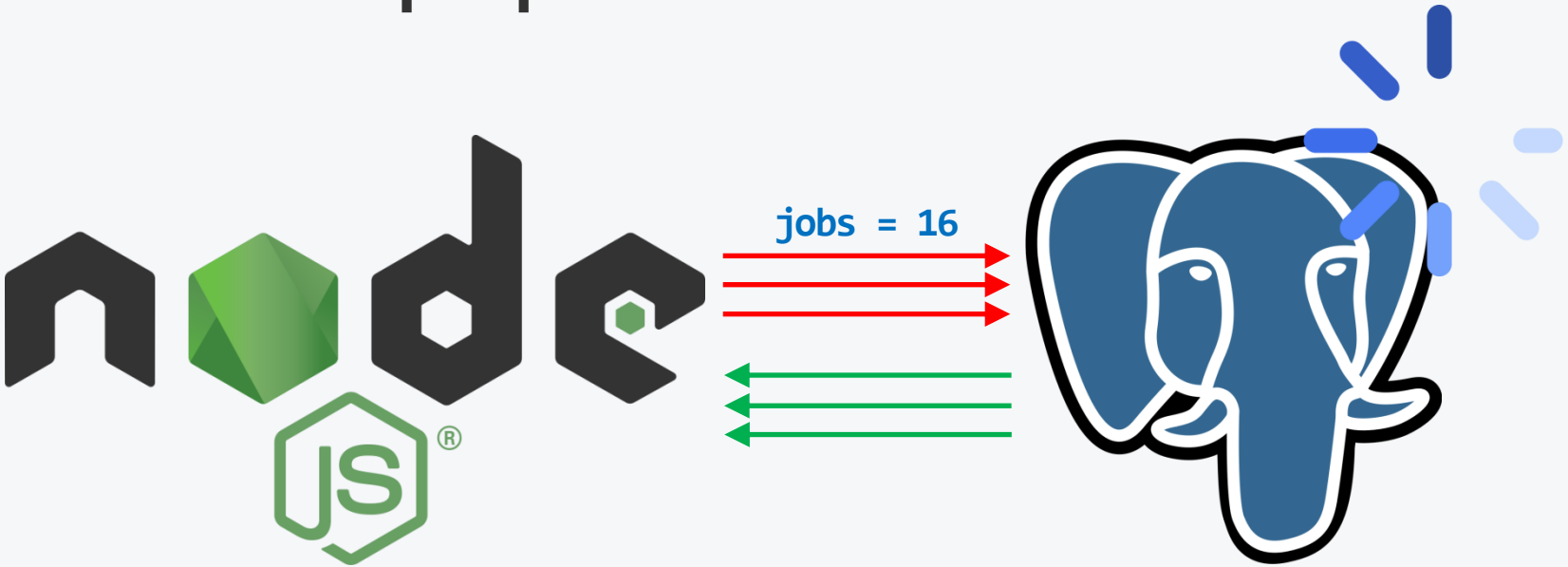
  отсутствие масштабирования, несоответствие лимитов контейнера

- полное исчерпание CPU

  проблемы с гипервизором?..

saby

Проблема на сервере



saby

Проблема на сервере

```
SELECT sum(i) FROM generate_series(1, 1e6) i; -- одно число
```

```
... LOG:  duration: 507.984 ms  plan:
  Query Text: SELECT sum(i) FROM generate_series(1, 1e6) i
  Aggregate  (cost=12.50..12.51 rows=1 width=32) (actual time=507.969..507.970 rows=1 loops=1)
    Buffers: temp read=2073 written=2073
    ->  Function Scan on generate_series i  (cost=0.00..10.00 rows=1000 width=32) (actual time=223.259..348.167 rows=1000000 loops=1)
          Buffers: temp read=2073 written=2073
```

~508ms для разового выполнения

saby

Проблема на сервере (CPU)

jobs = 16 vs. 4 core PostgreSQL

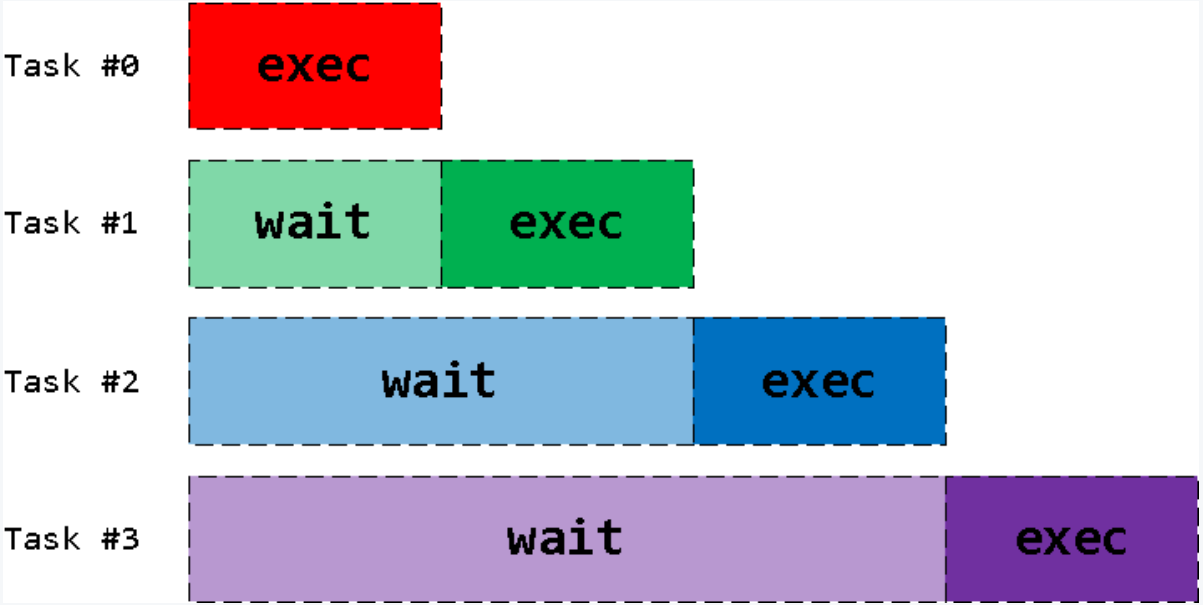
```
... 15:30:19.565 MSK [28936] LOG:  duration: 1767.403 ms  plan: ...
... 15:30:19.642 MSK [20360] LOG:  duration: 1859.703 ms  plan: ...
... 15:30:19.678 MSK [41036] LOG:  duration: 1947.635 ms  plan: ...
... 15:30:19.686 MSK [12032] LOG:  duration: 1938.924 ms  plan: ...
... 15:30:19.686 MSK [18300] LOG:  duration: 1939.397 ms  plan: ...
... 15:30:19.699 MSK [40900] LOG:  duration: 1917.059 ms  plan: ...
... 15:30:19.711 MSK [12728] LOG:  duration: 1965.203 ms  plan: ...
... 15:30:19.759 MSK [38540] LOG:  duration: 2027.971 ms  plan: ...
... 15:30:19.777 MSK [40276] LOG:  duration: 1992.891 ms  plan: ...
... 15:30:19.796 MSK [18644] LOG:  duration: 2064.386 ms  plan: ...
... 15:30:19.825 MSK [22932] LOG:  duration: 2043.790 ms  plan: ...
... 15:30:19.850 MSK [41732] LOG:  duration: 2087.980 ms  plan: ...
... 15:30:19.855 MSK [38024] LOG:  duration: 2093.014 ms  plan: ...
... 15:30:19.872 MSK [32480] LOG:  duration: 2141.214 ms  plan: ...
... 15:30:19.872 MSK [32544] LOG:  duration: 2109.777 ms  plan: ...
... 15:30:19.880 MSK [38348] LOG:  duration: 2118.152 ms  plan: ...
```

4x замедление

saby

Проблема на сервере (CPU)

```
jobs = 16 vs. 4 core PostgreSQL => 4 tasks : 1 core
```



saby

Проблема на сервере (CPU)

```
await client.query('SELECT pg_sleep(random() * 4)'); // разбежка до 4 сек
const res = await client.query('SELECT sum(i) FROM generate_series(1, 1e6) i');
```

```
... 16:12:23.203 MSK [23228] LOG:  duration: 579.658 ms  plan: ...
... 16:12:23.374 MSK [30036] LOG:  duration: 572.722 ms  plan: ...
... 16:12:23.483 MSK [33332] LOG:  duration: 617.321 ms  plan: ...
... 16:12:23.521 MSK [35592] LOG:  duration: 630.321 ms  plan: ...
... 16:12:23.827 MSK [38368] LOG:  duration: 611.399 ms  plan: ...
... 16:12:23.883 MSK [10440] LOG:  duration: 582.978 ms  plan: ...
... 16:12:23.929 MSK [35468] LOG:  duration: 562.947 ms  plan: ...
... 16:12:23.959 MSK [30792] LOG:  duration: 552.995 ms  plan: ...
... 16:12:24.398 MSK [19448] LOG:  duration: 539.472 ms  plan: ...
... 16:12:24.575 MSK [21936] LOG:  duration: 525.542 ms  plan: ...
... 16:12:24.607 MSK [40748] LOG:  duration: 533.656 ms  plan: ...
... 16:12:25.479 MSK [32416] LOG:  duration: 538.157 ms  plan: ...
... 16:12:25.566 MSK [11204] LOG:  duration: 531.848 ms  plan: ...
... 16:12:25.690 MSK [27716] LOG:  duration: 561.769 ms  plan: ...
... 16:12:26.128 MSK [34596] LOG:  duration: 550.124 ms  plan: ...
... 16:12:26.165 MSK [29528] LOG:  duration: 533.798 ms  plan: ...
```

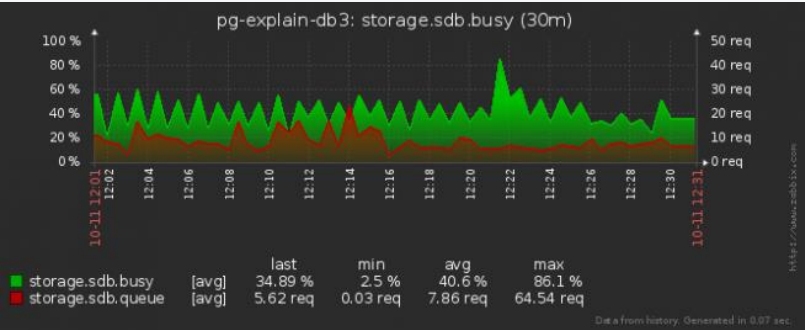
saby

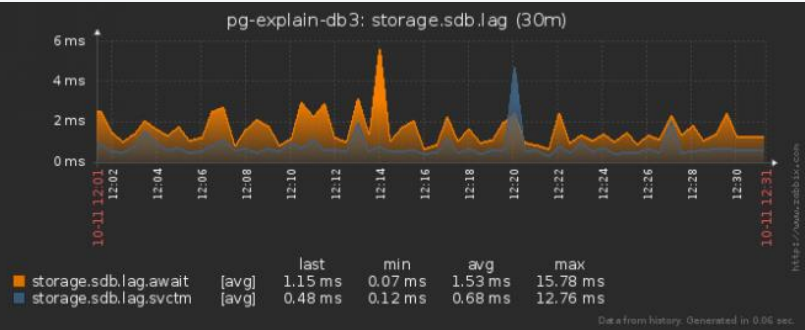
Проблема на сервере (CPU)

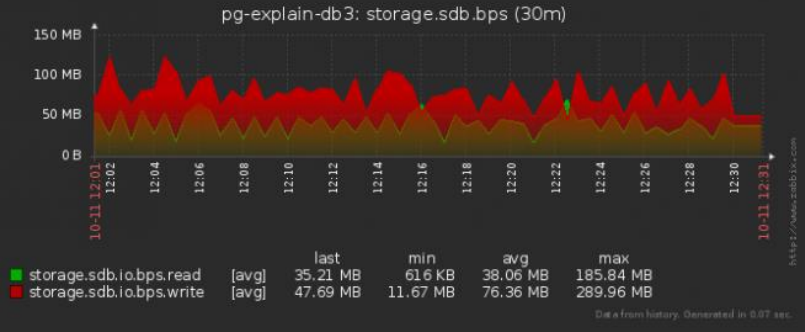
```
~~await client.query('SELECT pg_sleep(random() * 4)');~~
// ждем на клиенте, не занимая серверный процесс
await new Promise(resolve => setTimeout(resolve, 4 * 1000)); // разбежка до 4 сек
const res = await client.query('SELECT sum(i) FROM generate_series(1, 1e6) i');
```

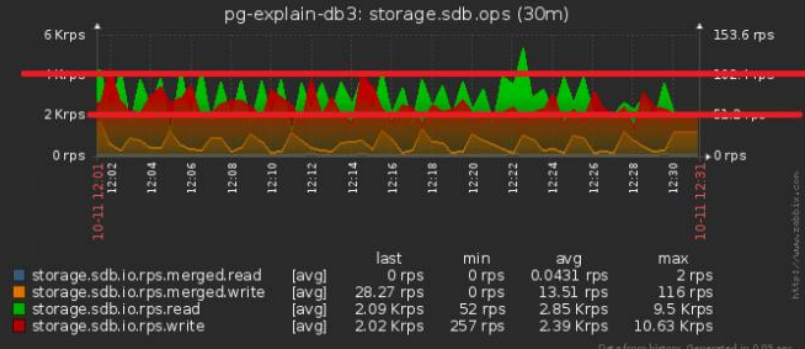
saby

Проблема на сервере (диск)









saby

Проблема на сервере (блокировки)

saby

Проблема на сервере

- избыточная синхронизация

  используйте принудительную де-синхронизацию

- исчерпание аппаратных ресурсов

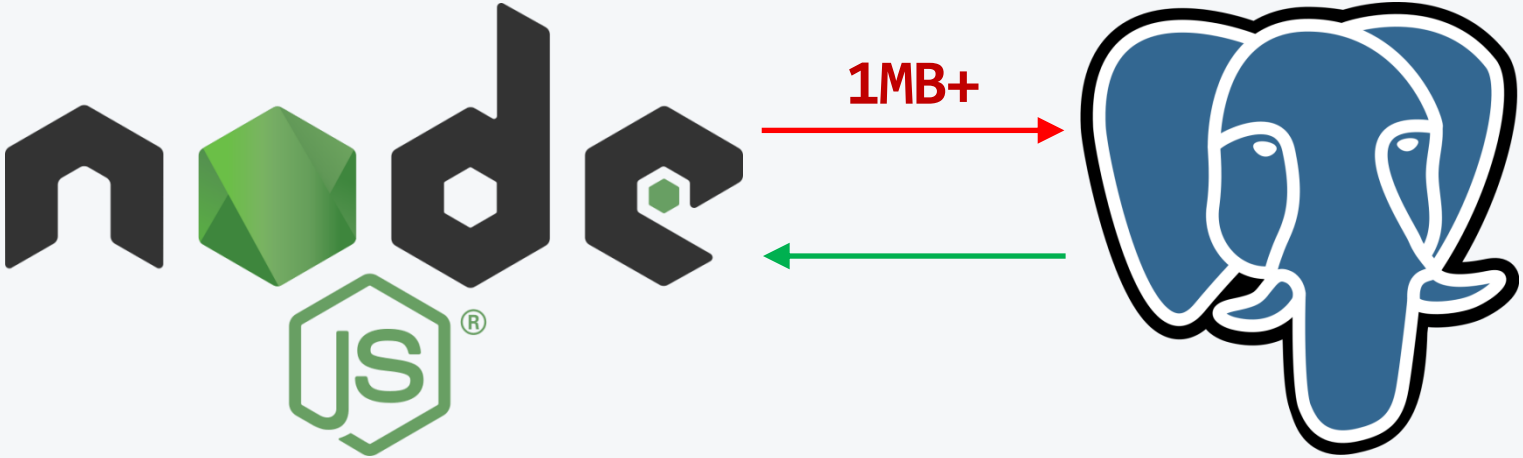
  мониторинг: CPU, RAM, swap(!), disk

- лимиты PostgreSQL

  количество подключений, предел блокировок

saby

Проблема с запросами



saby

Проблема с запросами (параметры в теле)

```
query = 'SELECT * FROM docs WHERE id IN (' + ids.join(',') + ')';
```

и тут **10K** идентификаторов

```
query = 'SELECT * FROM docs WHERE id = ANY($1::integer[])';
params = '{' + ids.join(',') + '}'; // текстовое представление массива
```

такой запрос уже можно в **PREPARED STATEMENTS**

saby

Проблема с запросами (multi-INSERT)

```
INSERT INTO users(id, name) VALUES(1, 'Vasya');
INSERT INTO users(id, name) VALUES(2, 'Petya');
INSERT INTO users(id, name) VALUES(3, 'Kolya');
```

```
INSERT INTO users(id, name)
  VALUES
    (1, 'Vasya')
  , (2, 'Petya')
  , (3, 'Kolya');
```

saby

Проблема с запросами (multi-INSERT)

```
INSERT INTO users(id, name)
  VALUES
    ($1, $2)
  , ($3, $4)
  , ($5, $6);
```

```
COPY users(id, name) FROM STDIN;
1\tVasya\n2\tPetya\n3\tKolya\n
\.
```

saby

Проблема с запросами (сгенерированный SELECT)

```
(
  SELECT * FROM tbl WHERE id = 1 ORDER BY ts DESC LIMIT 1
)
UNION ALL
(
  SELECT * FROM tbl WHERE id = 2 ORDER BY ts DESC LIMIT 1
)
UNION ALL
(
  SELECT * FROM tbl WHERE id = 3 ORDER BY ts DESC LIMIT 1
)
```

saby

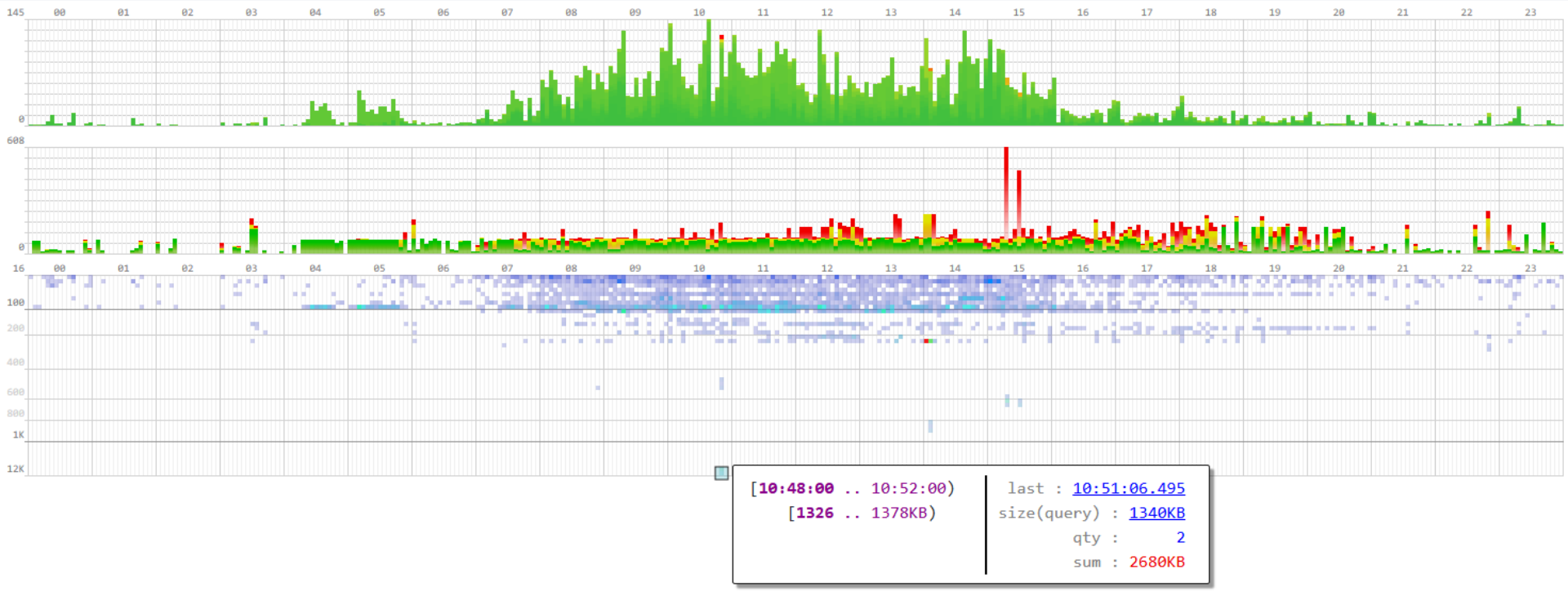
Проблема с запросами (сгенерированный SELECT)

```
SELECT
  T.*
FROM
  unnest('{1,2,3}'::integer[])
, LATERAL ( -- выполняется отдельно для каждого ID
    SELECT * FROM tbl WHERE id = unnest ORDER BY ts DESC LIMIT 1
  ) T;
```

PostgreSQL Antipatterns: «слишком много золота»

saby

Проблема с запросами



saby

Проблема с запросами

- мегазапросы

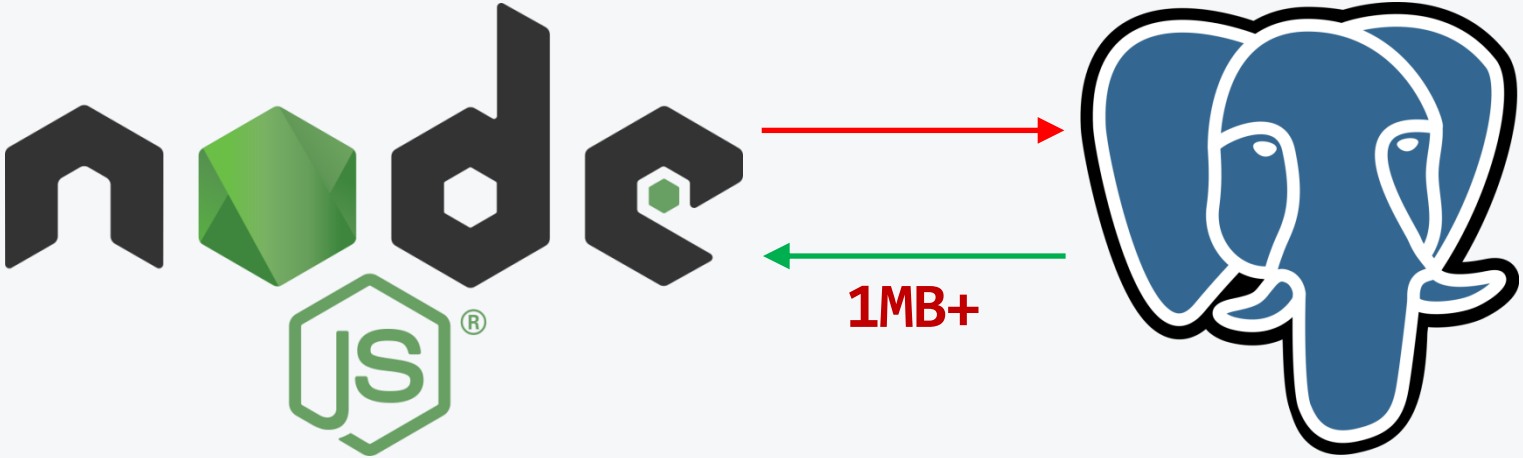
  PREPARED STATEMENT, LATERAL

- мегапараметры

  COPY, json-упаковка

saby

Проблема с ответами



saby

Проблема с ответами

```
SELECT i FROM generate_series(1, 1e6) i; -- миллион чисел
```

#	node, ms	tree, ms	rows	Function Scan on generate_series i	tm.rd	tm.wr
		365.479	1'000'000	итоговые результаты (31MB = rows=1'000'000 x width=32)	2'073	2'073
0	365.479		1'000'000 ▲	AT Function Scan on generate_series i (cost=0.00..10.00 rows=1'000 width=32) (actual time=227.908..365.479 rows=1'000'000 loops=1)	2'073	2'073
				Buffers: temp read=2'073 written=2'073	16MB	16MB

клиент получил **31MB** трафика!

на **365ms** мы заняли процесс (ядро CPU) на PG-сервере

«ширина» строки **32 байта** – потому что **numeric**!

saby

Проблема с ответами

```
... LOG:  duration: 737.459 ms  plan:
  Query Text: SELECT i FROM generate_series(1, 1e6) i
  Function Scan on generate_series i (cost=0.00..10.00 rows=1000 width=32) (actual time=225.135..362.437 rows=1000000 loops=1)
    Buffers: temp read=2073 written=2073
```

время получения ответа **на клиенте (~720ms)**

время выполнения запроса **на сервере (~360ms)**

сервер «не работает» половину времени!

31MB / 0.72s = ~340Mbps

3 «жадных» клиента – гигабит в утиль…

saby

Проблема с ответами (неэффективные типы)

```
SELECT i::integer FROM generate_series(1, 1e6) i;
```

привели к **достаточному** типу – сэкономили 4x трафика и 20% времени сервера

не надо передавать **uuid** в виде **text**

saby

Проблема с ответами (обработка на клиенте)

```
cur.execute("SELECT * FROM events;")
rows = cur.fetchall();
rows.sort(key=lambda row: row.ts, reverse=True);
limit = 26
print(rows[offset:offset+limit]);
```

извлекаем **все записи**

а их там миллион…

кто-то не знает **LIMIT … OFFSET …**

saby

Проблема с ответами (лишние поля)

```
docs <- SELECT * FROM doc WHERE owner = 'Alice';
SELECT sum(qty) FROM doc_item WHERE doc_id IN (${docs.map(({id}) => id).join(',')});
```

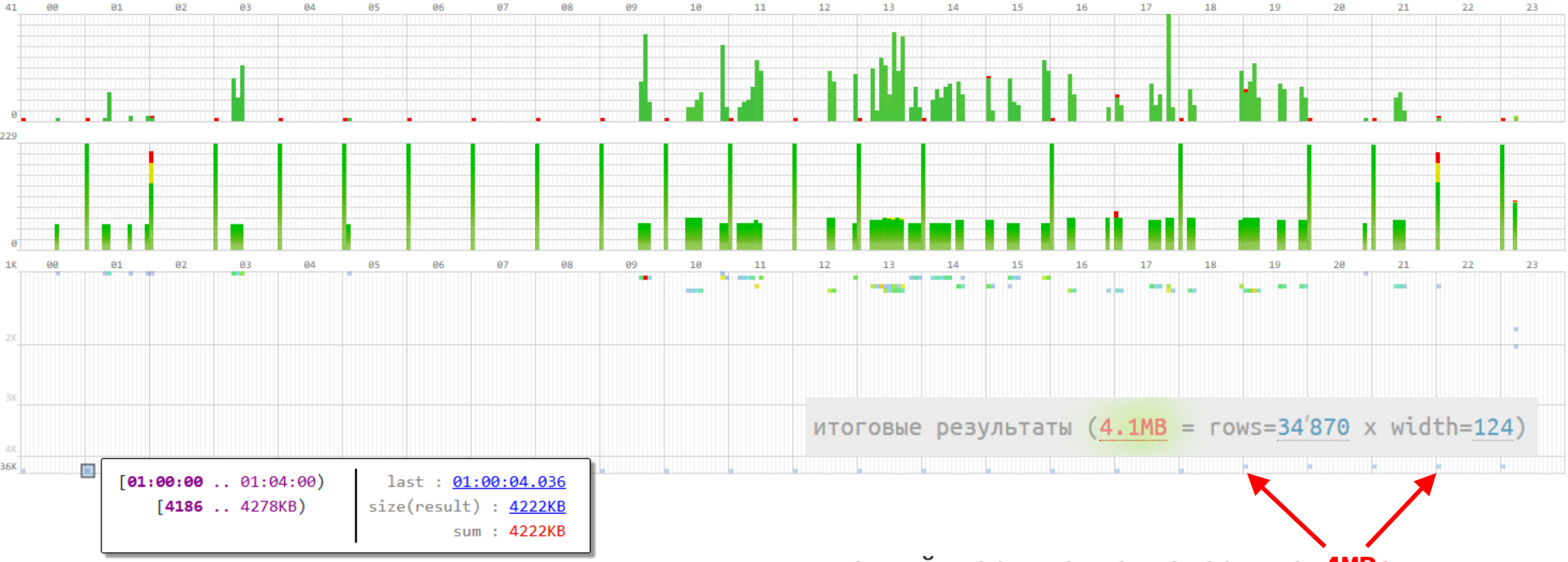
извлекаем **все поля**

а там blob со сканом…

кто-то не знает **вложенные запросы**

saby

Проблема с ответами



каждый час кто-то качает по **4MB+**

saby

Проблема с ответами

- «сложный» запрос лучше обработки на клиенте

  учим SQL-синтаксис и возможности PostgreSQL

- клиенту точно вот это все надо?

  контролируем получаемые клиентом записи, поля и их типы

- бессмысленные экспорт-импорты

  сравнить «все со всем» без ограничений срока

saby

Понять происходящее на сервере PostgreSQL

explain.saby.dev

saby

https://github.com/Kilor/Presentations

Появились вопросы?

Кирилл Боровиков

Компания «Тензор», технический директор

kilor@tensor.ru / explain.tensor.ru / explain.saby.dev